SP-14 Green Chess AI

CS 4850 (01) Spring 2026

4/12/2026 Sharon Perry

Test Plan

Purpose: Test and verify program functionality.

Revision History

Date	Version	Author	Description
4/11/2026	1.0	Group members	Working on the testing plan and outlining testing report
4/12/2026	2.0	Group members	Finished testing report

Table of Contents

1. Overview 4
1.1. Purpose 4
1.2. Scope 4

2. Testing Summary 5
2.1. Scope of Testing 5

3. Analysis of Scope and Test Focus Areas 6
3.1. Release Content 6
3.2. Regression Testing 6
3.3. Platform Testing 6

4. Progression Test Objectives 7

5. Regression Test Objectives 8

6. Other Testing 9
6.1. Security 9
6.2. Stress & Volume Testing (S&V) 9
6.3. Connectivity Testing (CT) 9
6.4. Disaster Recovery/Back Up 9
6.5. Unit Testing 9
6.6. Integration Testing 9

7. Test Strategy 10
7.1. Test level responsibility 10
7.2. Test Type & Approach 10
7.3. Build strategy 10
7.4. Test Execution Schedule 10
7.5. Facility, data, and resource provision plan 10
7.6. Testing Tools 11
7.7. Testing Handover Procedure 11
7.8. Testing Metrics 11

8. Test Environment Plan 12
8.1. Test Environment Management 12
8.2. Test Environment Details 12
8.3. Establishing Environment 13
8.4. Environment Control 13
8.5. Environment Roles and Responsibilities 13

9. Assumptions and Dependencies 14
9.1. Assumptions 14

9.2. Dependencies 14

10. Entry and Exit Criteria 15

11. Administrative Plan 16

11.1. Approvals 16

11.2. Test Milestones and Schedule 16

11.3. Training 16

11.4. Defect Management 16

12. Definitions 17

13. Test Report 18

14. References 19

15. Points of Contact 20

1. Overview

1.1. Purpose

The purpose of this document is to test the developed prototype to identify presence and severity of issues. This information will then be used to fix the issues in preparation for submission of the final product.

1.2. Scope

The scope of this document is to define what will be tested and how, as well as to document the test and the results of the tests. The document will define:

- What was tested
- How it was tested
- Results of the test and what changes are planned according to the results

2. Testing Summary

2.1. Scope of Testing

2.1.1. In scope

- What will be tested: crashing, illegal move (phasing), premature checkmate, issues with AI (static AI, major AI slowdown as depth increases, etc.)
- How testing will be performed:
    - By hand; testers will take previously discovered issues and see if those are still present, as well as try to push the program further to see if anything else breaks.
    - AI will be tested for being static by playing against it; the depth slowdown will require changing and testing the AI code.

2.1.2. Out of scope

Not applicable, no third-party testers

3. Analysis of Scope and Test Focus Areas

3.1. Release Content

Not applicable

3.2. Regression Testing

Test case for game launching properly
Test case for piece boundaries off canvas
Test case for move generation
Test case for promotion (to queen)

3.3. Platform Testing

Software:

- OS – Windows
- Hardware – someone's specs (?), but I don't think this is terribly important, since we're running chess.
- Software – Python 3.13, Pygame library

4. Progression Test Objectives

Ref	Function	Test Objective	Evaluation Criteria	X-Ref	P
1001	Choose_move	AI does not make the same moves every time	AI makes different moves at the beginning of the game	N/A	Medium
1002	negamax	Returns best most in reasonable amount of time	Best move is returned in less than 10 seconds	N/A	High

5. Regression Test Objectives

Ref	Function	Test Objective	Evaluation Criteria	X-Ref	P
0007	Get_moves	Returns all moves available for a color	All legal moves in a position are returned	N/A	Medium
0001	N/A	Program does not crash on launch	Program launched successfully without crashing	N/A	High
0008	Choose_move	AI player chooses move of best scoring	Best move out of the move set is returned	N/A	Medium
0002	N/A	Program does not crash when pieces are dragged outside of boundary	Program does not crash when picking up and dragging pieces outside of boundary of game window	N/A	High
0003	Moves_calc	No pieces through each other/illegal moves	Pieces that are unable to jump over pieces (anything that's not a knight) cannot move through other pieces	N/A	High
0004	Is_checkmate	No premature checkmates	The game continues until the king is out of moves	N/A	High
0006	restart	Restart button working	When the game ends, the restart button resets the game	N/A	Low

6. Other Testing

6.1. Security

Not applicable

6.2. Stress & Volume Testing (S&V)

Not applicable

6.3. Connectivity Testing (CT)

Not applicable

6.4. Disaster Recovery/Back Up

Not applicable

6.5. Unit Testing

Not applicable

6.6. Integration Testing

Not applicable

7. Test Strategy

7.1. Test level responsibility

Test Level	External Party	Project Team	Business
Unit Testing		P	
Integration Testing		P	
Security Testing		P	
Connectivity Testing		P	
User Acceptance Testing		P	
Production Verification Testing		P	

7.2. Test Type & Approach

Test Type	Objectives
Progression Requirements	Testing will be done after the presentation of the prototyped project. New implementations will function as defined within the test case.
Regression testing	Basic functionality, done before the presentation, does not alter in function before changes and after changes.

7.3. Build strategy

The build will be implemented in a one line installation of a git clone command and an execution through navigation to the SRC directory and running python main.py

7.4. Test Execution Schedule

Progression:

- We will play against improved AI several times and ensure that the AI will differ in moves in the same position.

Regression:

- Game launches without crashing
- Move generation will be logged to the console and manually checked
- Promotion will be tested by moving a pawn to the end of the board in PvP mode
- Restart will be tested through functionality of the R key

7.5. Facility, data, and resource provision plan

7.5.1. Test environment

Our game is the testing environment, testing will be done through manual input.

7.5.2. Access to other applications

Not applicable

7.5.3. Testing Requirements

Python 3.13

Pygames

Terminal to run the main file

7.5.4. Data Requirements

Not applicable

7.5.5. Resources & Skills

Not applicable

7.6. Testing Tools

Process	Tool
Test case creation	Word
Test case tracking	Word
Test case execution	Manual
Test case management	Word
Defect management	Not applicable

7.7. Testing Handover Procedure

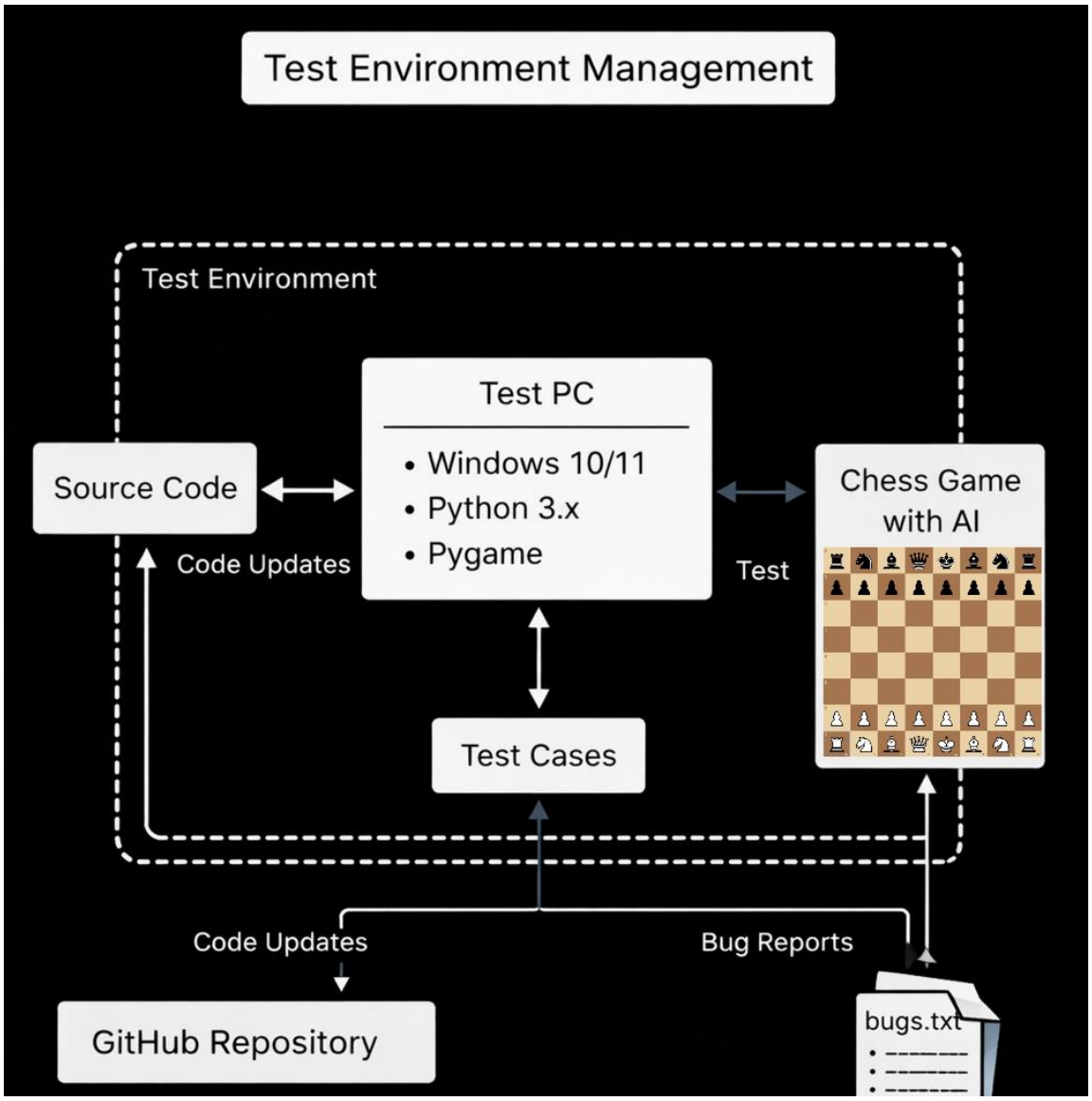
Not applicable

7.8. Testing Metrics

We will judge on a case by case basis, however, in general, if the program performs as expected, the test will be judged as “passed”, otherwise “fail”

8. Test Environment Plan

8.1. Test Environment Management



8.2. Test Environment Details

8.2.1. Testers

Vladimir Ryzhov

- Python, Windows OS
- System access not applicable

Kenneth Burke

- Python, Windows OS
- System access not applicable

8.2.2. Hardware and Firmware

Ensure it can run python

8.2.3. Software

Python 3.13
pygame

8.2.4. Interfaces

Not applicable

8.2.5. Other Materials

Not applicable

8.3. Establishing Environment

Task	Requirements	Responsibility	Start Date	End Date
Install Python	Required	All	4/11/2026	4/12/2026
Clone Repository	Required	All	4/11/2026	4/12/2026

8.4. Environment Control

Not applicable

8.5. Environment Roles and Responsibilities

Role	Staff Member	Responsibilities
Release Manager	Marc-henry Moise	Responsible for overall establishment, coordination and support of the test environment
Test Manager	Vladimir Ryzhov	Responsible for advising release manager of environment requirements for planning, establishment and ongoing
Project Manager	Kenneth Burke	Escalation point for testing issues.
Tester	Brett Talley	Responsible for test cases

9. Assumptions and Dependencies

9.1. Assumptions

Testers have knowledge of the codebase, testers know what modules they are testing and how to test for it prior and without training.

9.2. Dependencies

Python and Pygame

10. Entry and Exit Criteria

Phase 1: Does not crash on launch

- Entry: Launches
- Exit: Game does not crash

Phase 2: Does not crash on canvas boundary issues

- Pieces can be dragged off canvas boundary
- Game does not crash

Phase 3: Piece moves are legal and properly generated

- Legal piece moves show up
- Illegal piece moves do not show up

Phase 4: AI moves are legal

- AI makes legal moves
- AI continues to make legal moves

11. Administrative Plan

11.1. Approvals

NOT APPLICABLE

Task	Responsible Person	Escalation/ Approver
Systems Integration Signoff		
User Acceptance Testing Signoff		
Production Verification Testing Signoff		

11.2. Test Milestones and Schedule

Milestone	Planned End Date	Actual End Date	Resource
Phase 1 Criteria	4/12/2026	4/12/2026	
Phase 2 Criteria	4/12/2026	4/12/2026	
Phase 3 Criteria	4/12/2026	4/12/2026	
Phase 4 Criteria	4/12/2026	4/12/2026	

11.3. Training

NOT APPLICABLE

Training Requirement	Staff	Date

11.4. Defect Management

Not applicable, since no defect management tool will be used

12. Definitions

NOT APPLICABLE

Term/Acronym	Definition

13. Test Report

Test ID	Requirement	Pass	Fail	Severity	Details
0001	Lauch successfully	X		N/A	
0002	No boundary crashing		X	Moderate	Dragging pieces and trying to place them outside of left boundary causes the game to crash
0003	No pieces phasing through each other	X		N/A	
0004	No premature checkmate	X		N/A	
0006	Restart buttons working	X		N/A	
0007	Correct moves returned	X		N/A	
0008	AI returns best move	X		N/A	
1001	AI not making the same moves every time	X		N/A	
1002	Negamax functionality	X		N/A	Was able to make moves in less than 10 seconds

14. References

#	Document name	Version	Comments
	Assignment instructions	N/A	Reference to how document is filled out.
	The STP template	N/A	Outlines the document

15. Points of Contact

NOT APPLICABLE

Primary Contact	
Name	
Title/Organisation	
Phone	
Email	
Secondary Contact	
Name	
Title/Organisation	
Phone	
Email	